List Comprehensions Review

Q & A Practice

Lessons 2A & 2B

📋 Quick Syntax Reminder

```
# Basic
[expression for item in list]

# Filter (if at END) - removes items
[expression for item in list if condition]

# Transform (if-else at START) - keeps all items
[A if condition else B for item in list]
```

QUESTION 1

Convert to Comprehension

```
numbers = [1, 2, 3, 4, 5]

doubled = []
for n in numbers:
    doubled.append(n * 2)
```

Write this as a one-line comprehension

ANSWER 1

```
numbers = [1, 2, 3, 4, 5]

doubled = [n * 2 for n in numbers]

# Result: [2, 4, 6, 8, 10]
```

[n * 2 for n in numbers]

QUESTION 2

Create with range()

Use range() to create: [1, 4, 9, 16, 25]

(Hint: These are squares of 1-5)

ANSWER 2

```python
squares = [n ** 2 for n in range(1, 6)]

# Result: [1, 4, 9, 16, 25]
```

[n ** 2 for n in range(1, 6)]

QUESTION 3

What's the Output?

```
words = ["cat", "dog", "bird"]

result = [w.upper() for w in words]

print(result)
```

What prints?

ANSWER 3

```python
words = ["cat", "dog", "bird"]

result = [w.upper() for w in words]

print(result)  # ["CAT", "DOG", "BIRD"]
```

["CAT", "DOG", "BIRD"]

QUESTION 4

Convert to Comprehension (with Filter)

```
scores = [85, 60, 92, 55, 78, 95]

passing = []
for s in scores:
    if s ≥ 70:
        passing.append(s)
```

Write as a comprehension

ANSWER 4

```
scores = [85, 60, 92, 55, 78, 95]

passing = [s for s in scores if s ≥ 70]

# Result: [85, 92, 78, 95]
```

[s for s in scores if s >= 70]

Filter: `if` goes at the END

QUESTION 5

How Many Items?

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

result = [n for n in nums if n > 5]
```

How many items in result?

ANSWER 5

```python
nums = [1, 2, 3, 4, 5, 6, 7, 8]

result = [n for n in nums if n > 5]
# [6, 7, 8]

print(len(result))  # 3
```

3 items (filter removed 5 items!)

QUESTION 6

Write the Comprehension

From [10, 25, 8, 30, 15], keep only numbers > 12 AND double them

Expected: `[50, 60, 30]`

ANSWER 6

```
nums = [10, 25, 8, 30, 15]

result = [n * 2 for n in nums if n > 12]

# Result: [50, 60, 30]
```

[n * 2 for n in nums if n > 12]

Filter first (if at end), then transform (expression)

QUESTION 7

Convert to Comprehension (if-else)

```
temps = [65, 82, 70, 90, 75]

labels = []
for t in temps:
    if t ≥ 80:
        labels.append("hot")
    else:
        labels.append("cool")
```

Write as a comprehension

ANSWER 7

```
temps = [65, 82, 70, 90, 75]

labels = ["hot" if t ≥ 80 else "cool" for t in temps]

# Result: ["cool", "hot", "cool", "hot", "cool"]
```

["hot" if t >= 80 else "cool" for t in temps]

Transform: `if-else` goes at the START

QUESTION 8

How Many Items?

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

result = ["even" if n % 2 == 0 else "odd" for n in nums]
```

How many items in result?

ANSWER 8

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

result = ["even" if n % 2 == 0 else "odd" for n in nums]
# ["odd", "even", "odd", "even", "odd", "even", "odd", "even"]

print(len(result))  # 8
```

8 items (transform keeps ALL items!)

QUESTION 9

🚨 THE BIG QUESTION 🚨

Version A

```
[n for n in nums if n > 0]
```

Version B

```
[n if n > 0 else 0 for n in nums]
```

What's the difference between A and B?

ANSWER 9

A: FILTER

```
[n for n in nums if n > 0]
```

`if` at END

Removes items!

B: TRANSFORM

```
[n if n > 0 else 0 for n in nums]
```

`if-else` at START

Keeps all items!

Position determines behavior!

QUESTION 10

What's the Output?

```
names = ["", "alice", "", "bob", "charlie", ""]

result = [name for name in names if name]

print(result)
print(len(result))
```

What are the two lines of output?

ANSWER 10

```
names = ["", "alice", "", "bob", "charlie", ""]

result = [name for name in names if name]

print(result)        # ["alice", "bob", "charlie"]
print(len(result))   # 3
```

["alice", "bob", "charlie"] and 3

Empty strings are falsy - `if name` filters them out!

QUESTION 11

Write as a Loop

Convert this comprehension to a regular loop:

```
cubes = [x ** 3 for x in range(1, 5)]
```

Write the equivalent loop

ANSWER 11

```python
# Comprehension:
cubes = [x ** 3 for x in range(1, 5)]

# Loop version:
cubes = []
for x in range(1, 5):
    cubes.append(x ** 3)

# Result: [1, 8, 27, 64]
```

QUESTION 12

Create with range()

Use range() to create all even numbers from 2 to 20: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

ANSWER 12

```python
# Option 1: Use step in range
evens = [n for n in range(2, 21, 2)]

# Option 2: Use filter
evens = [n for n in range(1, 21) if n % 2 == 0]

# Result: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Both work! range(2, 21, 2) is more efficient

QUESTION 13

Choose the Right Tool

You need a list of ONLY the negative numbers (the actual values):

```
nums = [5, -3, 8, -1, 0, -7]
# Want: [-3, -1, -7]
```

Filter or Transform? Write the comprehension.

ANSWER 13

```
nums = [5, -3, 8, -1, 0, -7]

# Use FILTER - we want fewer items
negatives = [n for n in nums if n < 0]

# Result: [-3, -1, -7]
```

FILTER - because we want to remove items

QUESTION 14

Choose the Right Tool

You need to label EVERY number as "pos", "neg", or "zero":

```
nums = [5, -3, 8, -1, 0, -7]
# Want: ["pos", "neg", "pos", "neg", "zero", "neg"]
```

Filter or Transform? Write the comprehension.

ANSWER 14

```
nums = [5, -3, 8, -1, 0, -7]

# Use TRANSFORM - we want to keep all items
labels = ["pos" if n > 0 else "neg" if n < 0 else "zero"
          for n in nums]

# Result: ["pos", "neg", "pos", "neg", "zero", "neg"]
```

TRANSFORM - because we keep all items, just label them

QUESTION 15

What's the Output?

```
prices = [9.99, 24.99, 4.99, 15.00]

result = [f"${p:.2f}" for p in prices if p > 10]

print(result)
```

What prints?

ANSWER 15

```python
prices = [9.99, 24.99, 4.99, 15.00]

result = [f"${p:.2f}" for p in prices if p > 10]
# Filter: 24.99 and 15.00 pass
# Transform: format with $

print(result)  # ["$24.99", "$15.00"]
```

["$24.99", "$15.00"]

QUESTION 16

Fix the Bug

```
numbers = [1, 2, 3, 4, 5]

# This crashes! Why?
result = [n if n > 2 for n in numbers]
```

What's wrong and how do you fix it?

ANSWER 16

```
# BUG: "if" without "else" can't go at the start!

# If you want to FILTER:
result = [n for n in numbers if n > 2]
# [3, 4, 5]

# If you want to TRANSFORM:
result = [n if n > 2 else 0 for n in numbers]
# [0, 0, 3, 4, 5]
```

if at START needs else! | if at END = filter (no else)

QUESTION 17

Write as a Loop

Convert this comprehension to a regular loop:

```
result = ["big" if x > 10 else "small" for x in [5, 15, 8, 20]]
```

Write the equivalent loop

ANSWER 17

```python
# Comprehension:
result = ["big" if x > 10 else "small" for x in [5, 15, 8, 20]]

# Loop version:
result = []
for x in [5, 15, 8, 20]:
    if x > 10:
        result.append("big")
    else:
        result.append("small")

# Result: ["small", "big", "small", "big"]
```

QUESTION 18

Challenge!

From 1-30, get squares of numbers divisible by 5

Expected: `[25, 100, 225, 400, 625, 900]`
(5^2, 10^2, 15^2, 20^2, 25^2, 30^2)

ANSWER 18

```python
result = [n ** 2 for n in range(1, 31) if n % 5 == 0]

# Filter: 5, 10, 15, 20, 25, 30
# Transform: square each

# Result: [25, 100, 225, 400, 625, 900]
```

[n ** 2 for n in range(1, 31) if n % 5 == 0]

QUESTION 19

Quick Fire: What's the Length?

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Comprehension	Length?
`[x for x in data]`	?
`[x for x in data if x > 5]`	?
`["y" if x > 5 else "n" for x in data]`	?

ANSWER 19

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Comprehension	Length
`[x for x in data]`	**10** (basic)
`[x for x in data if x > 5]`	**5** (filter)
`["y" if x > 5 else "n" for x in data]`	**10** (transform)

Filter = fewer | Transform = same

QUESTION 20

Final Boss 🎮

Write ONE comprehension that:

1. Takes numbers 1-20
2. Keeps only odd numbers
3. Cubes each one

Expected: [1, 27, 125, 343, 729, 1331, 2197, 3375, 4913, 6859]

ANSWER 20

```
result = [n ** 3 for n in range(1, 21) if n % 2 == 1]

# Odd numbers 1-20: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
# Cubed: 1, 27, 125, 343, 729, 1331, 2197, 3375, 4913, 6859
```

[n ** 3 for n in range(1, 21) if n % 2 == 1]

📚 Key Takeaways

Basic: `[expression for item in list]`

Filter: `[expr for item in list if cond]` → **fewer items**

Transform: `[A if cond else B for item in list]` → **same items**

Position of "if" determines behavior!

Review Complete!

You're ready for the assessment!